



Introducción a Git

**El control de versiones que
cambió el mundo**

¿Control de versiones?

Es un software que **registra los cambios** realizados sobre archivos a lo largo del tiempo.

- Permite **volver a versiones anteriores** de un archivo o proyecto.
- Facilita el **trabajo en equipo** sin pisarse entre sí.
- Mantiene un **historial completo** de quién cambió qué y cuándo.



Decreto



Decreto final



Decreto final
este si



Decreto final
este si sisi es



Decreto final final



Decreto final
listo



Decreto final por fin



Decreto final por fin
espero que si



Decreto final
finalísimo



Decreto último
revisado



Decreto último
de las últimos



Decreto final
gracias a Dios

Breve historia

Año	Herramienta	Tipo
1972	SCCS	Local
1982	RCS	Local
1990	CVS	Centralizado
2000	SVN (Subversion)	Centralizado
2000	BitKeeper	Distribuido
2005	Mercurial	Distribuido
2005	Git	Distribuido

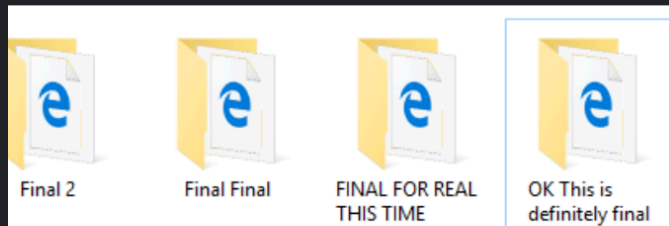
“ Los sistemas **centralizados** dependen de un servidor único. Los **distribuidos** le dan a cada desarrollador una copia completa del historial. ”



I prefer the real version control



I said the *real* version control



Nicolás Riedel | Introducción al Desarrollo de Software - Facultad de Ingeniería (UBA)

Perfection

¿Cómo nació Git?

- El kernel de **Linux** usaba un sistema propietario llamado **BitKeeper**.
- En 2005, la relación con BitKeeper se rompió.
- **Linus Torvalds** creó Git en pocas semanas con estos objetivos:
 - Velocidad
 - Diseño simple
 - Soporte para desarrollo no lineal (miles de ramas)
 - Completamente distribuido
 - Open source

Repositorio

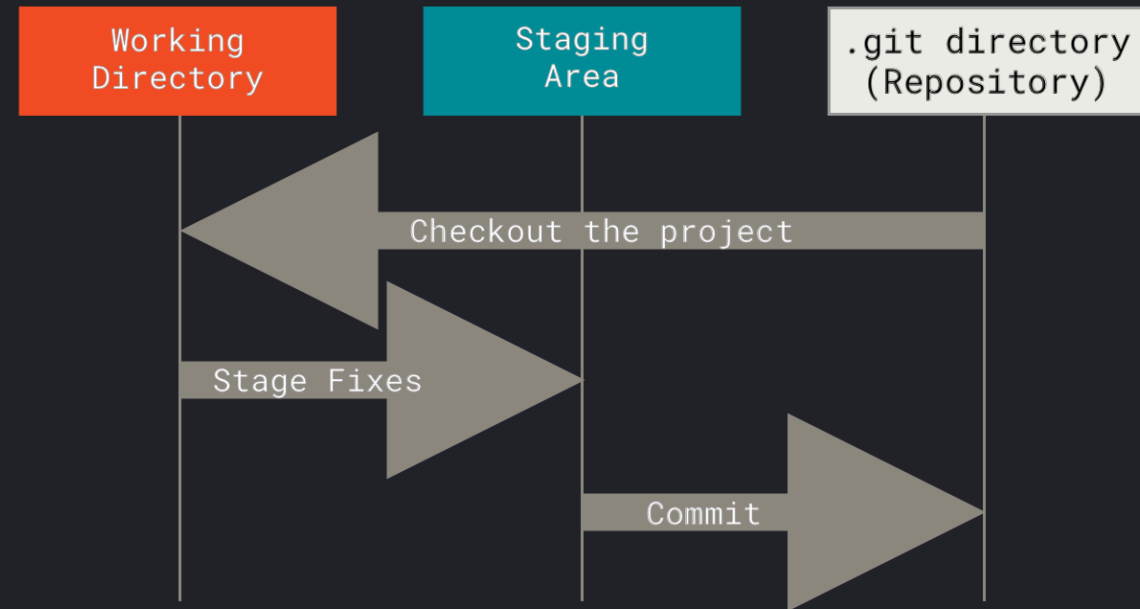
Un **repositorio** (repo) es un directorio cuyo historial de cambios está siendo trackeado (rastreado) por Git.

- Se inicializa con `git init`.
- Contiene una carpeta oculta `.git/` con toda la metadata.
- Puede ser **local** (en tu máquina) o **remoto** (en un servidor).

Los tres estados de Git

Todo archivo en un repositorio puede estar en uno de estos estados:

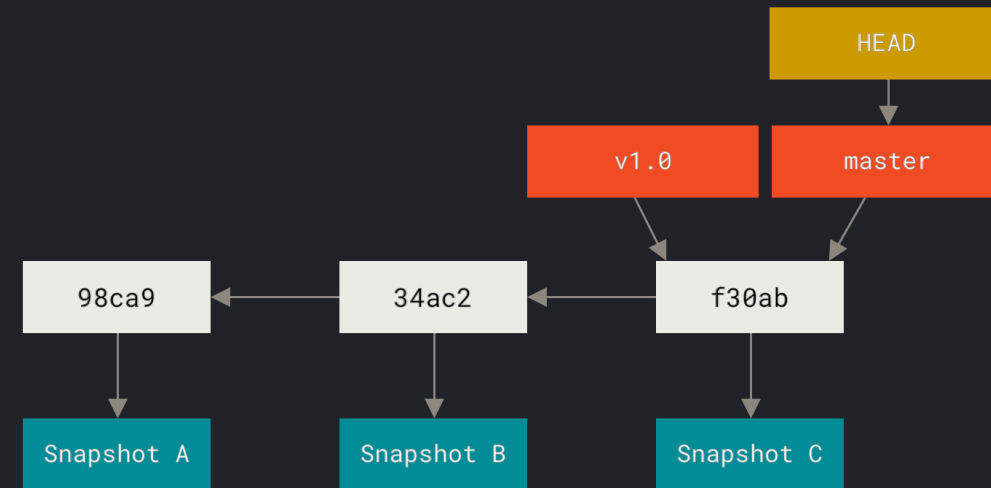
- **Working Directory** → donde editás tus archivos.
- **Staging Area (index)** → archivos preparados para el próximo commit.
- **Repository (.git)** → historial confirmado.



Ramas (branches)

Una **rama** es una línea independiente de desarrollo.

- Sirven para llevar distintas líneas de desarrollo en paralelo.
- Luego se pueden **mergear** (mezclar) para unificarse.
- Lo vamos a ver en profundidad la siguiente clase.



Flujo básico

```
Editar archivo → git add → git commit  
(Working Dir)  (Staging)  (Repository)
```

1. **Modificás** un archivo en tu directorio de trabajo.
2. **Agregás** los cambios al staging area con `git add`.
3. **Confirmás** los cambios con `git commit`.

Comandos básicos

Comando	Descripción
<code>git init</code>	Inicializa un repositorio nuevo
<code>git status</code>	Muestra el estado actual de los archivos
<code>git add <archivo></code>	Agrega cambios al staging area
<code>git commit -m "mensaje"</code>	Confirma los cambios con un mensaje
<code>git log</code>	Muestra el historial de commits
<code>git diff</code>	Muestra las diferencias no stageadas

Más comandos útiles

Comando	Descripción
<code>git clone <url></code>	Clona un repositorio remoto
<code>git pull</code>	Trae y fusiona cambios del remoto
<code>git push</code>	Sube commits al repositorio remoto
<code>git branch</code>	Lista o crea ramas
<code>git checkout <rama></code>	Cambia de rama
<code>git merge <rama></code>	Fusiona una rama en la actual

Hosting de repositorios

Git es **local**. Para colaborar necesitamos un servidor remoto.

Plataforma	Destacado
GitHub	La más popular, gran comunidad open source
GitLab	CI/CD integrado, se puede self-hostear
Bitbucket	Integración con Jira y el ecosistema Atlassian
Gitea	Liviano, self-hosted, open source

“

En esta materia vamos a usar **GitHub**.

”

Crear y subir un repositorio

```
# Crear un repositorio local
mkdir mi-proyecto
cd mi-proyecto
git init

# Crear un primer archivo y commit
echo "# Mi Proyecto" > README.md
git add README.md
git commit -m "Primer commit"
```

Conectar con GitHub y subir

```
# Agregar el remoto (usar la URL SSH de tu repo en GitHub)
git remote add origin git@github.com:usuario/mi-proyecto.git

# Subir los cambios
git push -u origin main
```

“ Primero creá el repositorio **vacío** en GitHub (sin README ni .gitignore). ”

GRACIAS!